Introduction to Computer Organisation and Architecture UCCD1133

Chapter 2

Number systems and Data Representation

Disclaimer

- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

Computer Codes

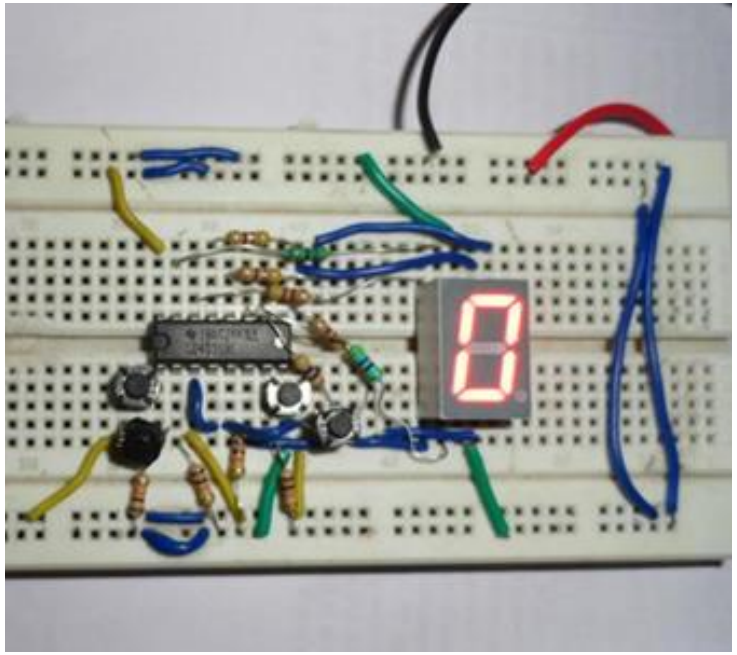
- A code is the use of a number system for representing information
- Binary system is widely used as codes to represent numbers, text, pictures, etc.
  - Since computers process all info in binary – the most efficient electronic form.
- Several types of well-known codes.
  - Codes to represent numeric.
  - Codes to represent character.
  - Codes for detecting and correcting errors.
  - Codes for serial data transmission and storage.

Binary Coded Decimal (BCD)

- BCD code is used to represent the decimal digits 0 - 9.
  - Always 4 bits.
  - BCD is a type of weighted code
  - Each bit position is weighted with 8, 4, 2, 1 from MSB to LSB
  - Hence, also called 8-4-2-1 code

DECIMAL DIGIT	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

- Note that 1010, 1011, 1100, 1101, 1110 and 1111 are not used. They are invalid in 8421 BCD code.
- IF the number contains two decimal digits, then it's equivalent BCD will be the respective eight binary of the given decimal number.
- Four for the first decimal digit and next four for the second decimal digit.
- Example of applications:
  - Easy interface between man machine –e.g: keypad inputs
  - Used to encode numbers for output to numerical displays
  - Used in processors that performs decimal arithmetic.
- BCD system is not binary system. They are not the same.



BCD to 7 segment display

Binary Coded Decimal (BCD)

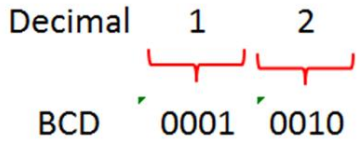
Example 15 : Find the equivalent BCD for the following decimal numbers

a) 12_{10}

b) 9750_{10}

Answer

a) $12_{10} = 00010010_{BCD}$



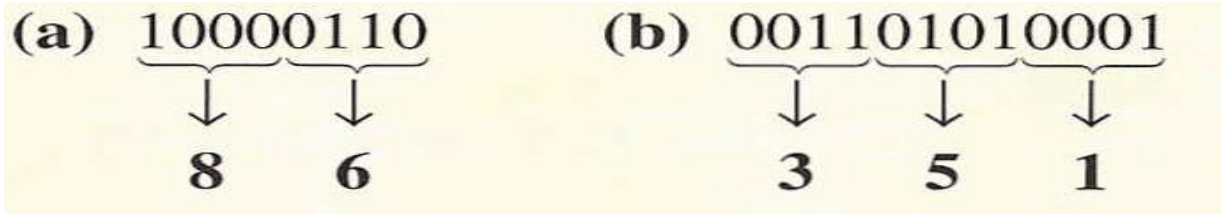
b) $9750_{10} = 1001011101010000_{BCD}$

Example 16 : Find the decimal numbers for the following BCD

a) 10000110_{BCD}

b) 001101010001_{BCD}

Answer



Binary Coded Decimal (BCD)

- BCD coding is easier

Decimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

$137_{10} = 000100110111_{BCD}$

The BCD code is 0001 0011 0111

$\frac{137}{2} = 68$ remainder 1
$\frac{68}{2} = 34$ remainder 0
$\frac{34}{2} = 17$ remainder 0
$\frac{17}{2} = 8$ remainder 1
$\frac{8}{2} = 4$ remainder 0
$\frac{4}{2} = 2$ remainder 0
$\frac{2}{2} = 1$ remainder 0
$\frac{1}{2} = 0$ remainder 1

The straight binary code is 10001001_2

Decimal number	Binary number	Binary Coded Decimal(BCD)
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	0001 0000
11	1011	0001 0001
12	1100	0001 0010
13	1101	0001 0011
14	1110	0001 0100
15	1111	0001 0101

Gray Code

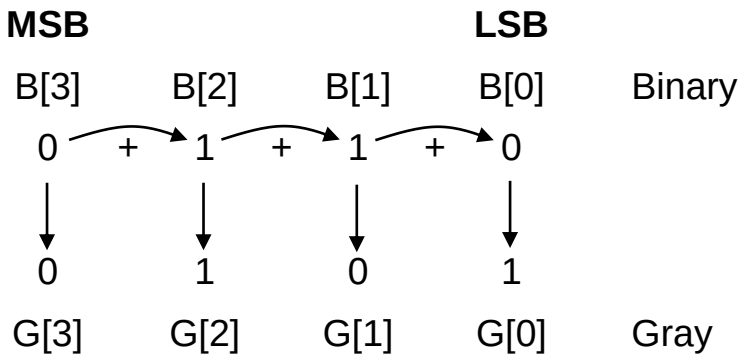
- Unweighted (no specific weight assigned to the bit position)
- Can't be used in arithmetic circuits
- Gray code important feature:
  - Its natural sequence exhibits a single bit change between successive code words.
- Useful for reducing error rate in data transfer applications
  - Lesser bit change while data is being transferring – chances of wrong bit transfer to the other side is lesser.

DECIMAL	BINARY	GRAY CODE
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Binary to Gray Code conversion

- The MSB in the Gray code is same as the corresponding MSB in the binary number
- From left to right, add each adjacent pair of binary code to get the next Gray code bit.
- Discard Carries

Example 17 : Convert the binary number 0110 to gray code

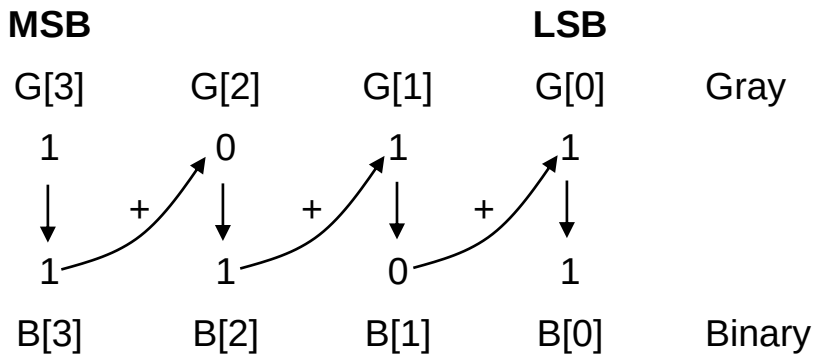


Gray Code 0101

Gray Code to binary conversion

- The MSB in the Binary number is same as the corresponding MSB in the Gray code
- From left to right, add each binary code bit generated to the gray code bit in the next adjacent position
- Discard Carries

Example 18: Convert the gray code 1011 to binary number



Binary 1101

Exercise Binary ⟷ Gray Code

Binary to Gray

- 1010
- 0110
- 0011
- 1001

Gray To Binary

- 1010
- 0110
- 0011
- 1001

Solution

Binary to Gray

- 1010 =1111
- 0110 =0101
- 0011 =0010
- 1001 =1101

Gray To Binary

- 1010 =1100
- 0110 =0100
- 0011 =0010
- 1001 =1110

ASCII (American Standard Code for Information Interchange)

- ASCII is a set of binary codes
- Used to represent the alphanumeric symbols.
- Widely used in data communication.
- An extra bit (parity bit) is attached to an ASCII code during transmission for error detection purpose

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

ASCII (American Standard Code for Information Interchange)

Example 19 : ASCII code representation of the word "Digital" is shown

Character	Binary Code	Hexadecimal Code
D	1000100	44
i	1101001	69
g	1100111	67
i	1101001	69
t	1110100	74
a	1100001	61
l	1101100	6C

Unsigned vs Signed number representation

- So far, we have considered only unsigned binary numbers that represent positive range of numbers.
- Sometimes programmers want to deal with both negative and positive range of numbers, which requiring a different binary number system.
- Signed binary numbers can be represented using:
  - Sign-magnitude number system.
  - Two's complement number system.
- You will probably deal with the terms:
  - zero extension (for unsigned numbers)
  - sign extension (for signed numbers)

Signed number –Sign Magnitude

- N-bit number correspond to decimal range $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$.
- The left-most bit is the sign bit and the remaining are the magnitude bit.
- The magnitude bits are true binary for positive and negative numbers.
- The left most bit in a binary number is the sign bit, which tells you whether the number is positive or negative.
- A "0"sign bit indicates a positive number.
- A "1"sign bit indicates a negative number.

Decimal Number System	Sign-Magnitude Number System (4-bit system)
+7	0111
+6	0110
+5	0101
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	1000
-1	1001
-2	1010
-3	1011
-4	1100
-5	1101
-6	1110
-7	1111
-8	-

2 possible representations of zero
- can complicate testing for zero - problems for programmers.

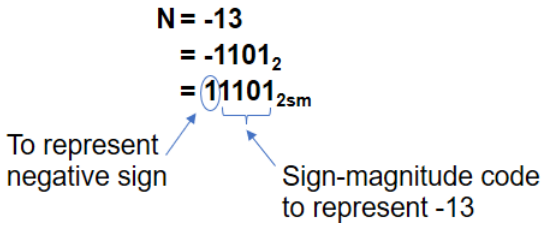
Signed number –Sign Magnitude

Example 20: The decimal number +23 and -23 is expressed as an 8-bit signed binary number using the sign-magnitude

Conversion	Signed (+ve)	Signed (-ve)
Signed Binary	00010111	10010111
	Sign bit magnitude bit	Sign bit magnitude bit
Decimal	23	-23

Example 21: Determine the binary sign-magnitude code for N = -13, expressed as a 5-bit binary number.

Answer:



- The implementation of the method is costly in terms of:
  - Circuitry (large arithmetic circuit)
  - Computation time (takes more time to produce the result).
- The disadvantage to have both +0 and -0 exist.
- Ordinary binary addition does not work. For example, $-5_{10} + 5_{10}$ gives $1101_2 + 0101_2 = 10010_2$
- Not popular in practice for representing signed numbers.

Signed number –2's complement

- 2's complement consists of:
  - Positive range $0 \leq N \leq (2^{n-1} - 1)$
    - ✓ Same as sign-magnitude number system
  - Negative range $-1 \geq N \geq (-2^{n-1})$
    - ✓ using complement of the negative numbers

- E.g. an 8-bit digital system can only operate on values between -128 to 127.

- Why use 2's complement?
  - Like binary system, natural numbering system for digital circuits.
  - The codes are arranged in such a way that is easy to build hardware to perform signed arithmetic operations (through complement arithmetic method).

Decimal Number System	Sign-Magnitude Number System	2's Complement Number System
+7	0111	0111
+6	0110	0110
+5	0101	0101
+4	0100	0100
+3	0011	0011
+2	0010	0010
+1	0001	0001
+0	0000	0000
-0	1000	-
-1	1001	1111
-2	1010	1110
-3	1011	1101
-4	1100	1100
-5	1101	1011
-6	1110	1010
-7	1111	1001
-8	-	1000

The MSB is sign bit.
0 => pos numbers.
1 => neg numbers.
For negative numbers, the rest of the bits DO NOT represent magnitude

E.g. does not represent -6

Signed Number 2's complement

- Algorithm to obtain the 2's complement of N:
  1. Find the binary equivalent of N.
  2. Complement each bit (that is, invert 0's to 1's and 1's to 0's).
  3. Add 1.

Example 22: Find the 2's complement of -2_{10} as a 4-bit number

Solution

1. $+2_{10}$ is 0010_2.
2. Inverting 0010_2 produces 1101_2.
3. $1101_2 + 1 = 1110_{2C}$.

Thus, **-2_{10} is 1110_{2C}**

Example 23: Given an 8-bit digital system, find the decimal number value of N = 1111 1010_{2c}. Also find the equivalent sign-magnitude representation.

Solution:

1. -6_{10}
2. The equivalent sign-magnitude is 10000110_{2sm}

Summary signed numbers

Sign –magnitude	
00010101	Decimal = + 21
10010101	Decimal = -21
1) Summing all the weights except the sign bit 2) The sign is determined by the sign bit	

2's complement	
00010101	Decimal = + 21
11101011	Decimal = -21
1) negative values are obtained by assigning a –ve to the weight of the sign bit 2) Summing all the weights	

Example 24: Determine the decimal values of the signed binary numbers:

	Sign -magnitude	2's complement
1. 00010101		
2. 10010101		
3. 11101010		
4. 11101011		
5. 00010111		

Solution:

	Sign -magnitude	2's complement
1. 00010101	21	21
2. 10010101	-21	-107
3. 11101010	-106	-22
4. 11101011	-107	-21
5. 00010111	23	23

Exercise

Convert the following decimal numbers to 6-bit two's complement binary numbers and add them.

1. $-26_{10} + 8_{10}$
2. $31_{10} + (-14_{10})$

Solution

1. $-26_{10} + 8_{10} = 100110 + 001000 = 101110$
2. $31_{10} + (-14_{10}) = 011111 + 110010 = 010001$

2's complement Arithmetic for Signed numbers: Overflow

- 2's complement uses complement arithmetic.
  - E.g. (A – B) can be performed by computing A + (-B) = A + B_{2c}
  - Easy hardware implementation.

- One problem of digital systems is that it have limited bits for data representation (limited range of signed values).
  - The decimal range for an n-bit digital system using 2 complement is $(-2^{n-1}) \leq N \leq (2^{n-1} - 1)$.
  - E.g. an 8-bit digital system can only operate on values between -128 to 127.

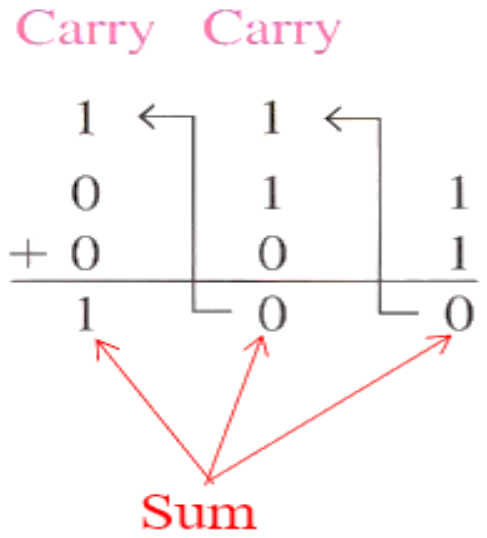
- If the result is out of the range, arithmetic overflow occurs.
  - The out-of-range result cannot be used.
    - ✓ Should design digital system that generates a warning signal
    - ✓ So that invalid numbers are not mistaken for correct results

Rule of thumb:
- Overflow can never occur for A = B – C.
- If two 2's complement numbers with the same sign are added, then overflow occurs if:
  - adding two positive numbers give a NEGATIVE result.
  - adding two negative numbers give a POSITIVE result.

Binary Arithmetic

Addition

$0 + 0 = 0$	Sum of 0 with a carry of 0
$0 + 1 = 1$	Sum of 1 with a carry of 0
$1 + 0 = 1$	Sum of 1 with a carry of 0
$1 + 1 = 10$	Sum of 0 with a carry of 1



Example:

$$\begin{array}{r} 11001 \\ +\ 1101 \\ \hline 100110 \end{array} \qquad \begin{array}{r} 111 \\ +\ 11 \\ \hline ??? \end{array}$$

Binary Arithmetic

Subtraction

$0 - 0 = 0$

$1 - 1 = 0$

$1 - 0 = 1$

$10 - 1 = 1$ 0-1 with a borrow of 1

A borrow is required in binary only when you try to subtract a 1 from a 0.

(a)

$$\begin{array}{r} 11 \\ -\ 01 \\ \hline 10 \end{array} \quad \rightarrow \quad \begin{array}{r} 3 \\ -\ 1 \\ \hline 2 \end{array}$$

Example:

$$\begin{array}{r} {}^{0}\not{1}^{1}01 \\ -\ \ 011 \\ \hline 010 \end{array}$$

Middle column: Borrow 1 from next column to the left, making a 10 in this column, then $10 - 1 = 1$.

Right column: $1 - 1 = 0$

Binary Arithmetic

Multiplication

$0 \times 0 = 0$

$0 \times 1 = 0$

$1 \times 0 = 0$

$1 \times 1 = 1$

(a)

$$\begin{array}{rl} & 11 \\ \times & 11 \\ \hline \text{Partial products} & 11 \\ + & 11 \\ \hline & \mathbf{1001} \end{array}$$

$$\begin{array}{r} 3 \\ \times 3 \\ \hline 9 \end{array}$$

Example:

Binary Multiplication : 101 x 111

$$\begin{array}{rl} & 111 \\ \times & 101 \\ \hline \text{Partial products} & 111 \\ & 000 \\ + & 111 \\ \hline & \mathbf{100011} \end{array}$$

$$\begin{array}{r} 7 \\ \times 5 \\ \hline 35 \end{array}$$

Binary Arithmetic

Division

Example

Perform the following binary division

a) $110 \div 11$

b) $110 \div 10$

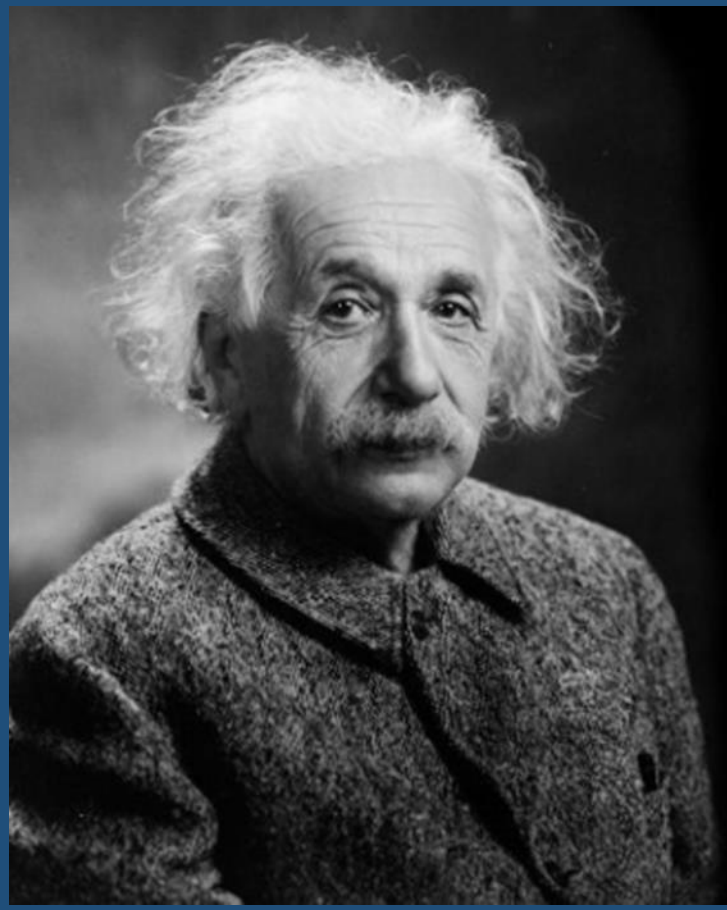
Solution:

(a)

$$\begin{array}{r} \mathbf{10} \\ 11\overline{)110} \\ \underline{11} \\ 000 \end{array} \qquad \begin{array}{r} 2 \\ 3\overline{)6} \\ \underline{6} \\ 0 \end{array}$$

(b)

$$\begin{array}{r} \mathbf{11} \\ 10\overline{)110} \\ \underline{10} \\ 10 \\ \underline{10} \\ 00 \end{array} \qquad \begin{array}{r} 3 \\ 2\overline{)6} \\ \underline{6} \\ 0 \end{array}$$



"Creativity is intelligence having fun"

– Albert Einstein

UTAR
UNIVERSITI TUNKU ABDUL RAHMAN

Q & A

Thank you